INTRODUCTION TO DBMS

Made By : **KASHIF SAYYAD**

MODULE I — DATABASE SYSTEM APPLICATION

■ Database, Database System Application and Purpose of Database System

A **database** is a collection of related data, where **data** means known facts that can be recorded and have an implicit meaning. For example the roll numbers, names and marks of all students of a college. A database plus software plus hardware plus users together is a **database system**; the set of programs that manages it is the **DBMS (Database Management System)**.

Database system application: wherever a large amount of permanent data needs to be shared and kept correct, databases are used — banking and finance, universities and colleges, airlines and railways (reservations), telecommunication, inventory and production, sales and human resources, library and e-commerce web sites.

Purpose of a database system: before databases, every application stored its data in flat **files**, which gave many problems — **redundancy and inconsistency** (the same data repeated at many places, and mismatched copies), **difficulty in accessing data** (a new report means writing a new program), **data isolation** (data scattered in different files and formats), **integrity problems** (rules were buried inside program code), **atomicity problems** (partial failure could leave half a transaction), **concurrent-access anomalies** and **security problems**. A database system removes all these problems with a small, central and shared program layer instead of a separate file-handling code in every application.

■ Advantages and Disadvantages of DBMS

Advantages of DBMS:

1. **Reducing redundancy** — data is kept at one central place instead of in every application's private files.
2. **Avoiding inconsistency** — since one copy is shared, all users see the same correct value.
3. **Sharing of data** — many users and applications get authorised access to the same data at the same time.
4. **Enforcing standards and data integrity** — rules such as not-null and check / unique are applied by the system itself.
5. **Security** — username / password and privilege levels restrict each user to his own data.
6. **Data independence** — the storage structure can be changed without changing application programs.
7. **Backup and recovery**, provision of **concurrency control**, and economies of scale.

Disadvantages of DBMS: high cost of the software itself, of conversion of old files and of additional hardware and trained staff; larger and more complex system; and for very small or very simple applications an ordinary file system may still be enough. Also, the performance of a general-purpose DBMS on some very specialised loads (like heavy embedded systems) may be poorer than a tuned file system.

■ Schema, Instances and Database State, Data Independence

The overall **design** of the database is called the **schema**; the detailed data stored at a particular moment is the **instance**. The schema rarely changes; the instance changes with every insert, update and delete. **Database state** means the actual data in the database at a moment of time — also called the current set of instances.

Data independence is the ability to change the schema at one level of the database system without changing the level above it. **Logical data independence** is the ability to change the conceptual schema without changing the external schemas. **Physical data independence** is the ability to change the internal schema (say storage structure, indexes) without any need to change the conceptual schema. Logical independence is harder to achieve than physical independence.

■ Database System Utilities

Database utilities are the standard auxiliary programs the DBMS gives to the DBA and to users: **Loading** (bulk transfer of initial data files into the database), **Backup** (making a copy of the whole database), **File reorganisation** (rearranging the physical storage to improve performance), **Performance monitoring** and statistics, and **user / privilege management** utilities. **Database Designers** design the schema; **casual and parametric end users**, system analysts and application program users are all consumers of these utilities.

■ Database System Architecture — Three Level Architecture

The **overall architecture** of a database system, as proposed by ANSI / SPARC, has three levels of data abstraction:

1. **Internal level (physical)** — the lowest level; describes how the data is actually stored (records, files, indexes, compression, storage allocation).
2. **Conceptual level (logical)** — describes **what data** is stored and the relationships among them; hides the physical detail; the whole community of users sees this single schema.
3. **External level (view)** — the highest level; many user **views**, each showing only that part of the database a given user group needs.

The levels are joined by **mappings**: external / conceptual and conceptual / internal. These mappings are exactly what provide logical and physical data independence: when one level changes, only the relevant mapping is adjusted and the levels above stay untouched.

MODULE II — DATABASE USERS AND ADMINISTRATORS, STRUCTURE OF DBMS

■ Database Users and Administrators — Responsibilities

Database users are of four broad kinds: **naive / parametric end users** who constantly query and update through canned transactions (bank tellers, booking clerks); **casual end users** who occasionally access through query

language; **sophisticated users** who write their own SQL and application programs; and **application programmers** who write programs in host languages with embedded DML calls.

The **database administrator (DBA)** is the person who has central control of the whole system. His **responsibilities** are: schema definition, storage structure and access-method definition, granting of authority (security), specifying integrity constraints, backing up and recovery, monitoring performance, tuning the system and acting as the mediator between users and the DBMS software.

■ Types of Data Model — Relational, E-R, Object-based; Hierarchical and Network Model

A **data model** is the collection of conceptual tools for describing data, their relationships, semantics and constraints. The **relational model** says the database is a collection of **tables (relations)**; the rows are **tuples** and the columns are **attributes**, and it uses the algebraic operations of relational algebra. The **E-R (entity-relationship) model** describes the data as entities, their attributes and the relationships among the entities — used mainly at the design stage. **Object-based data models** (object-oriented and object-relational) add identity, encapsulation, classes, inheritance and methods.

Hierarchical model organises the data as a **tree** where each record has only one parent and the relations are parent–child, needing an order of the records and pointers traversed top-to-bottom. **Network model** organises data as a **graph** where a record may have several parents; access runs along pointer chains. Both are old pre-relational models, connected by explicit pointers, and are still found in very old mainframe systems.

Phases of database design — (1) **Requirement collection and analysis** (what data and which operations the users need), (2) **Conceptual design** (high-level view of data as E-R diagram, independent of DBMS), (3) **Logical design** (conversion of the E-R diagram into the schema of the relational model) with the help of normalisation of the schema, and (4) **Physical design** (indexes, storage structures, file organisation, tuning).

■ E-R Model — Entity, Attributes and their Types, Entity Set, Entity Types

An **entity** is a real-world object with an independent existence, having some properties and distinguishable from other objects (a student, a course, an account). An **entity set** is the collection of all similar entities (all students). **Entity types**: a **strong entity** has its own key attribute and independent existence; a **weak entity** has no key of its own and is identified by belonging to a strong owner entity through a total, identifying relationship, marked by a double rectangle.

An **attribute** is a property of an entity or a relation. **Types: simple** (indivisible: roll) and **composite** (divide into parts: name into first + last); **single-valued** and **multi-valued** (phone numbers); **derived** (age can be derived from date of birth); and the **key attribute** which uniquely identifies each entity. **Naming conventions**: entity names in capital letters, attributes in lower case, relationship names in mixed case — and the names should be meaningful and unique in the enterprise schema.

Pictorial representation (symbols): **rectangle** for entity set, **diamond** for relationship set, **ellipse / oval** for attribute, underlined oval for key attribute, double ellipse for multi-valued, dashed ellipse for derived, **double rectangle** for weak entity, **double diamond** for identifying relationship, and **double lines** for total participation.

Data association means the mapping of attributes of one entity with the attributes of another entity through a relationship.

■ Data Association & Mapping Cardinalities, Constraints

Mapping cardinalities (cardinality ratios) tell how many entities of one set can be associated with an entity of the other set through a binary relationship: **one-to-one** (one citizen at most one passport), **one-to-many** (one department, many employees), **many-to-one** (many employees, one department) and **many-to-many** (many students enrol in many courses). The **mapping constraints** of cardinalities are declared in the schema (e.g. total participation, participation of every entity, keys and foreign keys).

- **Key constraint** — every relation must have a **primary key**: an attribute (or a combination) that is unique for every row and is never NULL (entity integrity).
- **Integrity (entity / domain) constraints** — every primary value must exist and be of correct data type and range.
- **Referential integrity (foreign key)** — a value of an attribute in one table must either be NULL or must equal a value of the primary key of some row in the referenced table; this keeps relationships consistent.
- **Unique constraint** — the column must not repeat values (e.g. email id).
- **Null / Not Null constraint** — decides whether the absence of value is permitted.
- **Domain constraint** — every value must be drawn from the declared data type of the column.
- **Check constraint** — a user-defined condition such as marks >= 0, age >= 0, gender in ('M','F').

■ DBMS versus RDBMS

Basis	DBMS	RDBMS
Data storage	as files, tables without strict relations	always in related tables (relations, tuples, attributes)
Keys	no fundamental key rule	every table must have a primary key
Redundancy	accepts duplicate rows	normalisation restricts redundancy
Constraints	mostly application-managed	referential integrity, constraints enforced by the system
Relation between tables	flat, independent	relationship via foreign keys
Codd rules	not necessary	must follow at least the Codd rules (e.g. 8+ / all 12 rules)
Examples	dBASE, FoxPro, FileMaker	Oracle, MySQL, SQL Server, PostgreSQL

MODULE III — DATA ASSOCIATION: NORMALIZATION, DDL AND DML

■ Normalization Techniques — 1NF, 2NF, 3NF and BCNF

Normalization is the systematic decomposition of big tables into smaller, better-structured relations to remove **redundancy** and the **update, insertion and deletion anomalies**. It is based on functional dependencies X → Y, meaning the value of X decides the value of Y.

1. **1NF (First Normal Form)** — every attribute contains only atomic (indivisible) values; no repeating groups or arrays inside a cell.
2. **2NF (Second Normal Form)** — the table is in 1NF and every non-key attribute is fully functionally dependent on the *whole* candidate key, i.e. no partial dependency on any part of a composite key.
3. **3NF (Third Normal Form)** — the table is in 2NF and no non-key attribute is transitively dependent on the primary key (no X → Y → Z chain of non-key attributes).
4. **BCNF (Boyce-Codd Normal Form)** — an advanced 3NF: for every functional dependency X → Y, X must be a superkey of the relation.

■ DDL — Data Definition Language, Creating Tables

DDL commands define or change the structure of the database objects themselves: **CREATE** (database / table / view / index), **ALTER** (add, modify or drop a column, add constraints), **DROP** (permanently destroy the table with all its data and structure), **TRUNCATE** (delete all rows but keep the structure) and **RENAME**. DDL statements are auto-committed.

```
CREATE TABLE student (
  roll_no   INT          PRIMARY KEY,
  name      VARCHAR(40)  NOT NULL,
  email     VARCHAR(50)  UNIQUE,
  sem       INT          DEFAULT 1,
  fees      NUMERIC(9,2) CHECK (fees >= 0)
);
ALTER TABLE student ADD city VARCHAR(20);
ALTER TABLE student MODIFY name VARCHAR(60);
DROP TABLE student;
```

■ DML — Inserting Data in Table, Select Statements

DML commands operate on the **data rows** of the existing structure: **INSERT** adds new rows, **SELECT** retrieves and displays rows, **UPDATE** changes the values of existing rows, **DELETE** removes rows; all of them can choose rows through the **WHERE** clause. DML changes are transaction-taking and can be confirmed by **COMMIT** or undone by **ROLLBACK**.

```
INSERT INTO student (roll_no, name, email, sem) VALUES (1,'Ali','ali@me.com',2);
SELECT * FROM student;
SELECT name, fees FROM student WHERE sem = 2 AND fees > 1000;
SELECT * FROM student WHERE name LIKE 'A%';
UPDATE student SET fees = fees + 500 WHERE roll_no = 1;
DELETE FROM student WHERE roll_no = 1;
SELECT * FROM student ORDER BY fees DESC LIMIT 3;
```

■ Restricting and Sorting Data, Agenda of Functions

Restricting rows is done through the **WHERE** clause with the operators = , <>, < , >, BETWEEN, IN, LIKE and IS NULL, joined by AND / OR / NOT. **Sorting the output** is done by the **ORDER BY** clause, which sorts ascending by default and descending by DESC; several keys can be sorted one after the other.

Conversion and conditional expressions convert a value of one type to another (CAST / CONVERT, TO_CHAR / TO_DATE) and give CASE ... WHEN ... THEN ... ELSE ... END to apply a condition inside a select list. **Aggregate (group) functions** work on a set of rows and return one value per group: **SUM(), AVG(), COUNT(), MIN(), MAX()**. **Grouping of rows**: **GROUP BY** divides the rows into groups on one or more columns, and **HAVING** puts a condition on the whole group (HAVING AVG(marks) > 60) — remember, WHERE filters *rows* but HAVING filters *groups*.

■ Displaying Data from Multiple Tables, Subqueries and Set Operators

When the columns required come from more than one table, the tables are joined in **FROM** with a **JOIN**: **equi / inner join** matches common keys (student NATURAL JOIN city on city_code), **outer joins** (LEFT / RIGHT / FULL) keep the unmatched rows with NULL filling, and **self join** joins a table with itself using aliases.

A **sub-query** is a SELECT inside another SELECT; it first produces an intermediate result used by the outer query, and can appear in WHERE, FROM (inline view) or HAVING. It can return a single value (scalar), a list (operator IN) or multiple columns. **Set operators** combine the results of two SELECTs having the same number and type of columns: **UNION** (common removal), **UNION ALL** (with duplicates), **INTERSECT** (common rows only) and **MINUS / EXCEPT** (rows of first query which are not in the second).
